

NumPy Exercises

for librosa audio analysis

15 progressive exercises using astronomy & space science data

01–04	05–06	07–08	09–11	12–15
Arrays & dtypes	Math & stats	Complex & linalg	2D matrices	Numerical methods

Datasets

NASA Exoplanet Archive 6 000+ confirmed exoplanets · mass, radius, orbital period, host star properties exoplanetarchive.ipac.caltech.edu · public domain Used in: Exercises 01–06, 08, 11–12, 14, 15	HYG Stellar Database v3 ~120 000 stars · position, magnitude, luminosity, color index, Cartesian coordinates astronexus.com/hyg · CC BY-SA 4.0 Used in: Exercises 07, 09–10, 13–15
---	--

No login · No API key · Direct `pd.read_csv()` load

How to use this document

Each exercise has two parts: the instructions (what to do) and the boilerplate (the data loading code already written for you). Copy the boilerplate into a cell, run it, then solve the numbered steps using only NumPy — the boilerplate handles all the pandas data loading so you can focus entirely on array operations.

The exercises are designed to be solved in order. Several later exercises reuse arrays built in earlier ones. When an exercise says 'reuse from Exercise N', it means keep that cell in your notebook and simply reference the variable.

One-time setup

```
pip install numpy pandas
```

That is all you need. NumPy and pandas are the only dependencies. No librosa required until you transfer these skills to the librosa exercises.

Mapping to librosa exercises

NumPy ex	Core concept	Directly used in librosa exercise
01–03	Shape, dtype, slicing	Ex 1–4: inspecting audio and STFT arrays
04	Boolean masking	Ex 3–6: selecting freq bins, filtering silence
05–06	axis=, broadcasting	Ex 8: MFCC normalization per coefficient
07	Complex numbers	Ex 5: STFT phase extraction and reassembly
08	dot, norm, cosine sim	Ex 10: manual vector similarity
09–10	2D indexing, argmax	Ex 11–12: self/cross-similarity matrices
11	vstack, diff, concat	Ex 9: delta MFCC stacking
12	log, clip, amplitude→dB	Ex 3: amplitude_to_db()
13	Full similarity matrix	Ex 11–13: cross_similarity()
14	DTW cost matrix + path	Ex 14: librosa.sequence.dtw()
15	End-to-end pipeline	Ex 15: full analysis figure

01

Tier 1 — Arrays & dtypes

Array shape, dtype, and basic inspection

NumPy concepts used

```
np.ndarray .shape .ndim .dtype .size len() np.nan np.isnan() np.sum(~np.isnan())
```

Objective

Load real exoplanet data and inspect the NumPy arrays that back it — their shape, type, and how many values are missing. This mirrors how you'll first inspect any *librosa* output array.

Instructions

1. Run the boilerplate to load the exoplanet dataset. You now have several 1D NumPy arrays (mass, radius, period, etc.).
2. For each of the arrays mass, radius, and period, print: `.shape`, `.ndim`, `.size`, and `.dtype`.
3. Count how many values in each array are NaN using `np.isnan()` and `np.sum()`. Then compute the percentage missing.
4. Use `len()` on one array and compare it to `.shape[0]` and `.size`. Confirm they all agree.
5. Try changing the dtype when creating an array: convert `dist` to `int32` using `.astype(np.int32)`. Print a few values and notice what happened to the decimal precision.

Dataset & boilerplate

Why this data fits

Real multi-column numeric data with natural NaNs (not all exoplanet parameters are measured for every planet) — perfect for practicing the first inspection steps you'll do on every *librosa* feature matrix.

Arrays you'll work with

- `mass` — 1D float array, planet masses
- `radius` — 1D float array, planet radii
- `period` — 1D float array, orbital periods
- `dist` — 1D float array, system distances

Boilerplate — NASA Exoplanet Archive — confirmed exoplanets

Run this once at the top of your notebook. All exercises in Tier 1 use these arrays.

```

import numpy as np

import pandas as pd

# NASA Exoplanet Archive – TAP API, no login required
# Returns confirmed exoplanet composite parameters as CSV
TAP = 'https://exoplanetarchive.ipac.caltech.edu/TAP/sync'
QUERY = (
    '?query=select+pl_name,pl_masse,pl_rade,pl_orbper,'
    'pl_orbsmax,pl_orbeccen,st_teff,st_rad,st_mass,'
    'st_lum,sy_dist,disc_year'
    '+from+pscomppars'
    '+where+pl_masse+is+not+null'
    '+and+pl_rade+is+not+null'
    '&format=csv'
)

df_exo = pd.read_csv(TAP + QUERY)

# Strip any comment rows NASA prepends (start with '#')
df_exo = df_exo[~df_exo.iloc[:, 0].astype(str).str.startswith('#')]
df_exo = df_exo.reset_index(drop=True)

# Convert key columns to NumPy arrays – NaNs preserved as np.nan
mass    = df_exo['pl_masse'].to_numpy(dtype=float)    # planet mass (Earth masses)
radius  = df_exo['pl_rade'].to_numpy(dtype=float)    # planet radius (Earth radii)
period  = df_exo['pl_orbper'].to_numpy(dtype=float)   # orbital period (days)
sma     = df_exo['pl_orbsmax'].to_numpy(dtype=float) # semi-major axis (AU)
ecc     = df_exo['pl_orbeccen'].to_numpy(dtype=float) # orbital eccentricity
t_eff   = df_exo['st_teff'].to_numpy(dtype=float)    # host star temperature (K)
st_rad  = df_exo['st_rad'].to_numpy(dtype=float)     # host star radius (solar radii)
st_mas  = df_exo['st_mass'].to_numpy(dtype=float)    # host star mass (solar masses)
st_lum  = df_exo['st_lum'].to_numpy(dtype=float)     # host star luminosity (log
solar)

dist    = df_exo['sy_dist'].to_numpy(dtype=float)    # system distance (parsecs)
yr      = df_exo['disc_year'].to_numpy(dtype=float)  # discovery year

```

02

Tier 1 — Arrays & dtypes

Array creation and arange / linspace

NumPy concepts used

`np.zeros()` `np.ones()` `np.full()` `np.arange()` `np.linspace()` `np.zeros_like()` `np.empty()`

Objective

Learn to manufacture arrays of specific shapes and value ranges — the foundation for creating time axes, frequency bins, and placeholder arrays in librosa work.

Instructions

6. Create a 1D array of 50 evenly-spaced values between 0 and 1 using `np.linspace()`. Print the first and last three values.
7. Create an integer array from 1 to 500 in steps of 5 using `np.arange()`. Print its shape and last value.
8. Create a zero-filled array with the same shape as `mass` using `np.zeros_like()`. Confirm shape and dtype match.
9. Use `np.full()` to create an array of 100 elements all set to `np.nan`. This is the pattern for pre-allocating a result buffer.
10. Simulate a 'number of exoplanets discovered per year' array: use `np.arange(1995, 2025)` as year labels and `np.zeros(30)` as a count buffer. Then fill counts by iterating over the `yr` array from the boilerplate.

Dataset & boilerplate

Why this data fits

The discovery year array gives a meaningful use case for building a range array (years) and a zero buffer (counts), then filling it — exactly the pattern for building feature vectors manually.

Arrays you'll work with

- `yr` — discovery years (use as reference values)
- `year_range` — `arange()` output
- `counts` — `zeros()` buffer to fill

Boilerplate — NASA Exoplanet Archive — same load as Ex 01

Reuse the arrays loaded in Exercise 01. No extra download needed.

```
# Reuse boilerplate from Exercise 01

# The yr array (discovery years) is key here:

# Year axis for the histogram you'll build
year_range = np.arange(1995, 2025)          # shape (30,)
counts      = np.zeros(len(year_range))      # shape (30,), all 0.0

# You'll fill counts in step 5 — one approach:
# for i, y in enumerate(year_range):
#     counts[i] = np.sum(yr == y)

# Or vectorized (try this after learning boolean indexing in Ex 04):
# counts = np.array([(yr == y).sum() for y in year_range])
```

03

Tier 1 — Arrays & dtypes

Indexing and slicing

NumPy concepts used

```
array[i]    array[i:j]    array[::step]    array[-1]    array[:, col]    2D array indexing
np.vstack() np.column_stack()
```

Objective

Master the slice notation that you'll use constantly in librosa to extract specific time frames, frequency bins, or coefficient rows from feature matrices.

Instructions

- From the mass array, extract: the first 10 values, the last 10 values, and every 100th value. Print each slice's shape.
- Stack mass, radius, and period into a 2D array with shape (N, 3) using `np.column_stack()`. Call it features.
- From features, extract: the entire first row (one planet), the entire second column (all radii), and the sub-matrix rows 50 to 100.
- Use the `[:, col]` pattern to pull column 0 (masses) back out of features and confirm it equals the original mass array using `np.array_equal()` on the non-NaN subset.
- Simulate a librosa-style operation: treat features as a (3, N) matrix by transposing it with `.T`. Then extract 'frame 200' as `features.T[:, 200]` and print its shape — it should be (3,), like slicing one time frame from an MFCC matrix.

Dataset & boilerplate

Why this data fits

The (N, 3) features matrix mirrors the shape structure of MFCC and chroma matrices. Practicing column extraction and transposition here makes the librosa [:, frame_index] pattern feel natural.

Arrays you'll work with

- `features` — (N, 3) float64 matrix
- `features_T` — (3, N) transposed view
- `mass, radius, period` — source columns

Boilerplate — NASA Exoplanet Archive — same load as Ex 01

Reuse the arrays loaded in Exercise 01.

```
# Reuse boilerplate from Exercise 01

# Build the 2D features matrix used in steps 2-5:

features = np.column_stack([mass, radius, period]) # shape (N, 3)
print('features shape:', features.shape)

# Transposed version — shape (3, N), like a librosa MFCC matrix
features_T = features.T # shape (3, N)
print('transposed shape:', features_T.shape)
```

04

Tier 1 — Arrays & dtypes

Boolean indexing and filtering

NumPy concepts used

`array[condition]` `np.where()` `np.isnan()` `&` | `~` operators `np.isfinite()` `np.argwhere()`

Objective

Filter arrays by condition without loops — the NumPy skill you'll use most when cleaning librosa output or selecting frames above an energy threshold.

Instructions

16. Find all planets with mass less than 2 Earth masses using a boolean mask. Print how many there are and their average radius.
17. Find planets where both mass AND radius are less than 2 (super-Earths). Use the `&` operator to combine two conditions.
18. Remove all NaN values from the mass array using `mass[np.isfinite(mass)]`. Compare the length before and after.
19. Use `np.where()` to classify planets: return 'small' where `radius < 1.6` and 'large' otherwise. Print the first 10 labels.
20. Find the indices of all planets with orbital period shorter than 1 day (ultra-short period planets) using `np.argwhere()`. Print how many exist and print the mass array values at those indices.

Dataset & boilerplate

Why this data fits

NaN filtering with `isfinite()` is a required first step in any `librosa` pipeline (silence detection, zero-energy frames). Boolean masking with combined conditions maps directly to selecting frequency ranges in a spectrogram.

Arrays you'll work with

- `valid` — boolean mask, finite in all three columns
- `m_clean` — filtered mass array
- `r_clean` — filtered radius array
- `p_clean` — filtered period array

Boilerplate — NASA Exoplanet Archive — same load as Ex 01

Reuse the arrays from Exercise 01. All filtering operations are purely NumPy — no pandas.

```
# Reuse boilerplate from Exercise 01

# Remove NaNs from mass and radius before most comparisons:

valid = np.isfinite(mass) & np.isfinite(radius) & np.isfinite(period)
m_clean = mass[valid]
r_clean = radius[valid]
p_clean = period[valid]

print(f'Clean subset: {valid.sum()} planets')
print('m_clean shape:', m_clean.shape)
```


05

Tier 2 — Math & statistics

Aggregations along axes

NumPy concepts used

```
np.mean()    np.std()    np.min()    np.max()    np.median()    np.percentile()    axis= parameter
np.nanmean()
```

Objective

Compute statistics across entire arrays and along specific axes of 2D arrays — the core of MFCC normalization and similarity threshold calculations in librosa.

Instructions

21. Compute mean, std, min, max, and median of the clean mass array. Print each with two decimal places using f-strings.
22. Build the features_T matrix (3 rows × N cols) from Ex 03. Use `np.mean(features_T, axis=1)` to get one mean per row (per property). Confirm the output shape is (3,).
23. Then compute `np.mean(features_T, axis=0)` and confirm the output shape is (N,). Explain in a comment which axis collapses which dimension.
24. Use `np.nanmean()`, `np.nanstd()` on the original (un-cleaned) mass array. Confirm the result matches what you get from the cleaned version.
25. Compute the 5th, 50th, and 95th percentiles of the period array using `np.percentile()`. Then count how many planets fall between p5 and p95.

Dataset & boilerplate

Why this data fits

The `axis=1` / `axis=0` distinction is the single most common source of confusion in librosa. This exercise drills it using physical arrays (3 properties × N planets) that mirror the (n_mfcc × frames) shape you'll encounter constantly.

Arrays you'll work with

- `features_T` — (3, N_clean) — one row per property
- `mass/radius/period` clean versions already available

Boilerplate — NASA Exoplanet Archive — same load as Ex 01

Reuse the clean arrays and features_T matrix. The `axis=` parameter is the main concept to internalize here.

```

# Reuse boilerplate from Exercise 01

# Rebuild the transposed features matrix:

valid = np.isfinite(mass) & np.isfinite(radius) & np.isfinite(period)
features_T = np.row_stack([
    mass[valid],
    radius[valid],
    period[valid],
]) # shape (3, N_clean)

print('features_T shape:', features_T.shape) # e.g. (3, 3200)

# axis=1 collapses columns -> shape (3,) one stat per row/property
# axis=0 collapses rows -> shape (N,) one stat per planet/column

```

06

Tier 2 — Math & statistics

Broadcasting and vectorized arithmetic

NumPy concepts used

broadcasting rules (N,) vs (N,1) vs (1,N) keepdims=True array - scalar array / array
 np.expand_dims()

Objective

Understand why subtracting a (3,1) array from a (3,N) array works — the exact operation used to normalize MFCCs per coefficient in librosa Exercise 8.

Instructions

26. Subtract the per-row mean from features_T using the result of np.mean(features_T, axis=1). This will fail. Read the error, then fix it by adding keepdims=True.
27. Standardize features_T by subtracting the mean and dividing by the std, both computed with axis=1, keepdims=True. Call it features_norm.
28. Verify the result: np.mean(features_norm, axis=1) should be all ~0 and np.std(features_norm, axis=1) should be all ~1.
29. Try the same using np.expand_dims() instead of keepdims=True. Confirm you get the same result.
30. Now standardize along axis=0 instead (per-planet normalization rather than per-property). What does the result mean physically? Write a short comment.

Dataset & boilerplate

Why this data fits

This is the exact broadcasting pattern used in librosa Exercise 8 (MFCC normalization). The intentional error in step 1 is the most instructive part — it teaches you to read broadcasting error messages, not just avoid them.

Arrays you'll work with

- `features_T` — (3, N) source matrix
- `row_means` — (3, 1) after `keepdims=True`
- `features_norm` — (3, N) normalized result

Boilerplate — NASA Exoplanet Archive — same load as Ex 01

Reuse `features_T` from Exercise 05. This exercise is deliberately about making a subtraction fail and then fixing it — that failure is instructive.

```
# Reuse features_T from Exercise 05 (shape: 3 × N_clean)

# Step 1: This WILL raise a broadcasting error — try it intentionally:
# row_means = np.mean(features_T, axis=1) # shape (3,)
# bad = features_T - row_means           # ERROR: (3,N) - (3,) doesn't broadcast

# Fix: keepdims=True preserves the (3,1) shape so broadcasting works:
row_means = np.mean(features_T, axis=1, keepdims=True) # shape (3,1)
row_stds  = np.std(features_T, axis=1, keepdims=True)  # shape (3,1)

# Now (3,N) - (3,1) broadcasts correctly:
features_norm = (features_T - row_means) / row_stds    # shape (3,N)
print('norm shape:', features_norm.shape)
```

07

Tier 2 — Math & statistics

Complex numbers and polar form

NumPy concepts used

`complex dtype` `np.abs()` `np.angle()` `np.exp()` `1j` `np.allclose()` `np.real()` `np.imag()`

Objective

Build and decompose complex-valued arrays — the exact operations used in librosa Exercise 5 (STFT phase extraction and reassembly).

Instructions

31. Create a small complex array by combining stellar luminosity and color index into a synthetic complex signal: $Z = s_lum + 1j * s_ci$ (after filtering both for finiteness). Print its dtype.
32. Extract the magnitude using `np.abs(Z)` and the phase using `np.angle(Z)`. Print the shapes and first three values of each.
33. Reconstruct the original complex array using the polar form: $Z_rebuilt = np.abs(Z) * np.exp(1j * np.angle(Z))$. This is the exact formula used in librosa Exercise 5.

34. Verify the reconstruction with `np.allclose(Z, Z_rebuilt)`. Also test `np.allclose(Z.real, Z_rebuilt.real)` and `np.allclose(Z.imag, Z_rebuilt.imag)` separately.
35. Intentionally break the reconstruction: replace magnitude with `np.abs(Z) * 2` (double it) and reassemble. What happens to the real and imaginary parts? This is what 'zeroing out frequency bins' does to the STFT.

Dataset & boilerplate

Why this data fits

Stellar luminosity (real part) and color index (imaginary part) make a physically meaningful complex array — a star's position on the HR diagram encoded as a complex number. The magnitude = distance from origin, the angle = direction in luminosity-color space.

Arrays you'll work with

- `s_lum` — stellar luminosity (real component)
- `s_ci` — color index B-V (imaginary component)
- `Z` — synthetic complex array for reconstruction drill

Boilerplate — HYG Stellar Database — star luminosity and color index

This exercise uses the HYG dataset. Run its boilerplate below, then use `s_lum` and `s_ci`.

```

import numpy as np

import pandas as pd

# HYG stellar database v3 - ~120 000 stars
# Hipparcos + Yale Bright Star + Gliese catalogs, CC BY-SA 4.0
HYG_URL = (
    'https://raw.githubusercontent.com/astronexus/'
    'HYG-Database/main/hyg/v3/hygdata_v3.csv'
)
df_hyg = pd.read_csv(HYG_URL)

# Remove stars with bogus distance (>= 100000 parsecs = missing parallax)
df_hyg = df_hyg[df_hyg['dist'] < 100000].reset_index(drop=True)

# Key NumPy arrays
s_dist = df_hyg['dist'].to_numpy(dtype=float)    # distance (parsecs)
s_mag = df_hyg['mag'].to_numpy(dtype=float)      # apparent magnitude
s_absmag = df_hyg['absmag'].to_numpy(dtype=float) # absolute magnitude
s_ci = df_hyg['ci'].to_numpy(dtype=float)        # color index B-V
s_lum = df_hyg['lum'].to_numpy(dtype=float)      # luminosity (solar units)
s_x = df_hyg['x'].to_numpy(dtype=float)          # Cartesian x (parsecs)
s_y = df_hyg['y'].to_numpy(dtype=float)          # Cartesian y (parsecs)
s_z = df_hyg['z'].to_numpy(dtype=float)          # Cartesian z (parsecs)
s_ra = df_hyg['ra'].to_numpy(dtype=float)        # right ascension (hours)
s_dec = df_hyg['dec'].to_numpy(dtype=float)       # declination (degrees)

print(f'Loaded {len(s_dist)} stars after filtering')
print('dist shape:', s_dist.shape, '| dtype:', s_dist.dtype)

# Build the synthetic complex array from stellar properties:
valid_ci = np.isfinite(s_lum) & np.isfinite(s_ci)
Z = s_lum[valid_ci] + 1j * s_ci[valid_ci]

print('Z dtype:', Z.dtype)          # should be complex128

```

08

Tier 2 — Math & statistics

Linear algebra: dot products and norms

NumPy concepts used

`np.dot()` `np.linalg.norm()` `cosine similarity` `np.inner()` `vector slicing [:, i]` `np.outer()`

Objective

Manually compute Euclidean distance and cosine similarity between two vectors — what librosa's `cross_similarity()` does internally, and what Exercise 10 in the librosa series requires you to do by hand.

Instructions

36. Take two specific 'planet feature vectors' from the `features_T` matrix: column 0 (planet 0) and column 1 (planet 1). Each is a length-3 vector.
37. Compute the Euclidean distance between them: `np.linalg.norm(a - b)`.
38. Compute the cosine similarity: `np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))`. Print the result and explain what it means.
39. Now normalize both vectors (divide each by its own norm) and compute `np.dot()` again. Confirm this equals the cosine similarity you computed in step 3.
40. Build a small 5×5 cosine similarity matrix: for all pairs among the first 5 planets in `features_T`, compute cosine similarity. Store results in a (5,5) matrix using `np.zeros((5,5))` and a nested loop. Then verify the diagonal is all 1s.

Dataset & boilerplate

Why this data fits

This directly parallels librosa Exercise 10 where you manually compute cosine similarity between MFCC frames. The 3-element planet feature vector is easier to reason about than a 13-element MFCC vector, but the math is identical.

Arrays you'll work with

- `features_norm` — (3, N) normalized matrix
- `a, b` — (3,) individual planet vectors
- `sim_matrix` — (5,5) cosine similarity scratch matrix

Boilerplate — NASA Exoplanet Archive — same load as Ex 01

Reuse the normalized `features_T` from Exercise 06 for cleaner results (already zero-mean, unit-std).

```
# Reuse features_norm from Exercise 06 (shape: 3 × N_clean)
# Each column is a planet's feature vector

a = features_norm[:, 0] # planet 0, shape (3,)
b = features_norm[:, 1] # planet 1, shape (3,)

print('Vector a:', a)
print('Vector b:', b)
print('Shapes:', a.shape, b.shape)

# Cosine similarity formula:
#  $\text{cos\_sim} = \frac{\text{dot}(a, b)}{\text{norm}(a) * \text{norm}(b)}$ 
```

09

Tier 3 — 2D arrays & matrices

2D array construction and imshow-ready matrices

NumPy concepts used

```
np.zeros((r,c))   np.reshape()   np.stack()   matrix[row, col]   matrix[:n, :]
np.fill_diagonal()   np.triu()   np.tril()
```

Objective

Build and manipulate 2D arrays in the shapes that librosa outputs — (frequencies × frames) and (n_coefficients × frames) — and understand what each dimension means.

Instructions

41. Build a (100, 50) zero matrix using `np.zeros()`. This represents 100 frequency bins × 50 time frames. Print shape and `ndim`.
42. Stack five rows from the HYG dataset (`s_mag`, `s_absmag`, `s_ci`, `s_lum`, `s_dist` — all finite) into a (5, N) matrix using `np.row_stack()`. This is analogous to stacking 5 MFCC coefficients.
43. Extract a 'submatrix' of the first 200 frames: `mat[:, :200]`. Extract a row (one coefficient over all frames): `mat[2, :]`. Print both shapes.
44. Build a 10×10 self-similarity matrix for the first 10 columns of the star matrix. Use `np.dot(A.T, A)` after normalizing columns. Then use `np.fill_diagonal()` to confirm the diagonal is 1s.
45. Experiment with `np.triu()` and `np.tril()` on your similarity matrix. In librosa similarity matrices are symmetric — the upper and lower triangles should be mirrors.

Dataset & boilerplate

Why this data fits

Five stellar properties stacked as rows produces a (5, N) matrix with the exact same shape logic as a (5-coefficient, N-frame) MFCC matrix. The self-similarity matrix built in step 4 is the direct analog of librosa Exercise 11.

Arrays you'll work with

- `mat` — (5, N_valid) stellar feature matrix
- `mat[:, :200]` — first 200 'frames' (stars)
- `mat[2, :]` — all values for property row 2

Boilerplate — HYG Stellar Database — multiple stellar properties

Run the HYG boilerplate once if you haven't yet. This exercise uses five stellar properties stacked into a matrix.


```

# Reuse HYG boilerplate from Exercise 07 if already loaded.
# Otherwise run it again here.

# Filter all five properties to rows where ALL are finite:
valid_all = (np.isfinite(s_mag) & np.isfinite(s_absmag) &
             np.isfinite(s_ci) & np.isfinite(s_lum) &
             np.isfinite(s_dist))

mat = np.row_stack([
    s_mag[valid_all],
    s_absmag[valid_all],
    s_ci[valid_all],
    s_lum[valid_all],
    s_dist[valid_all],
]) # shape (5, N_valid)

print('mat shape:', mat.shape) # e.g. (5, 90000)

```

10

Tier 3 — 2D arrays & matrices

Advanced indexing: argmax, argsort, and argwhere

NumPy concepts used

`np.argmax()` `np.argmin()` `np.argsort()` `np.argwhere()` `axis=` on 2D arrays `fancy indexing`
`mat[idx_array]`

Objective

Use index-returning functions to find, sort, and select rows — the operations you need to trace paths through similarity matrices and locate peaks in spectrogram data.

Instructions

46. Find the index of the brightest (lowest mag) star in `s_mag` using `np.argmin()`. Use that index to look up its distance, color index, and luminosity from the other arrays.
47. Sort all stars by distance using `np.argsort(s_dist)`. Extract the 10 nearest stars by using the sorted indices as fancy indices into `s_dist`, `s_mag`, and `s_absmag`.
48. On the 2D stellar matrix `mat` from Exercise 09: use `np.argmax(mat, axis=1)` to find, for each property row, which star (column index) has the highest value. Print the 5 indices.
49. Use `np.argmax(mat, axis=0)` to find, for each star, which property has the highest value. Print the first 10 results and reason about what they mean.

50. Simulate the 'worst country per year' path from the matplotlib exercises: on a submatrix `mat[:, :500]` (5 properties, 500 stars), use `np.argmax(axis=0)` to get one dominant-property index per star. These are the column indices; the row indices are 0–499. This is the same logic as overlaying a path on a heatmap.

Dataset & boilerplate

Why this data fits

`argmax` along `axis=0` on a 2D matrix is exactly the operation used to trace the 'worst country per year' path in the matplotlib exercises, and it's the same logic used to find the dominant frequency bin at each time frame in a spectrogram.

Arrays you'll work with

- `s_mag, s_dist, s_absmag` — 1D source arrays for fancy indexing
- `sort_idx` — `argsort` output, used as fancy index
- `mat[:, :500]` — submatrix for `axis=0` `argmax` exercise

Boilerplate — HYG Stellar Database — same load as Ex 09

Reuse the HYG arrays and `mat` matrix from Exercise 09. All operations here are pure index arithmetic.

```
# Reuse HYG arrays from Exercise 07/09
# Reuse mat (5 × N_valid) from Exercise 09

# Fancy indexing pattern (used in steps 1 & 2):
# idx = np.argmin(s_mag)          # single index
# s_dist[idx]                    # lookup in another array

# Sorted indices pattern (step 2):
# sort_idx = np.argsort(s_dist)   # shape (N,), ascending
# nearest_10 = sort_idx[:10]     # first 10 = closest stars
# s_mag[nearest_10]              # fancy index
```

11

Tier 3 — 2D arrays & matrices

Stacking, concatenation, and reshaping

NumPy concepts used

```
np.concatenate()  np.vstack()  np.hstack()  np.stack(axis=)  np.reshape()
np.ravel()        .flatten()
```

Objective

Combine and reshape arrays — used in `librosa` when assembling multi-feature matrices, concatenating deltas with base MFCCs, or preparing data for DTW.

Instructions

51. Split the exoplanet mass array into two halves using array slicing. Rejoin them with `np.concatenate()`. Confirm the result equals the original.
52. Stack `mass_clean`, `radius_clean`, and `period_clean` (1D arrays of the same length) into a (3, N) matrix using `np.vstack()`. Then add a 4th row (eccentricity) using `np.vstack([existing_matrix, new_row])`.
53. Append a 'delta' row to your matrix: compute the row-wise difference between consecutive columns using `np.diff(matrix, axis=1)`. Note the shape changes by 1 in the columns dimension. This is exactly what `np.gradient` and `librosa.feature.delta()` do.
54. Reshape a (3, 100) submatrix into (300,) using `.ravel()`, then back to (3, 100) using `np.reshape()`. Confirm values are unchanged with `np.array_equal()`.
55. Use `np.stack([mat_a, mat_b], axis=0)` to stack two (3, N) matrices into a (2, 3, N) array. Print the shape and explain what the three dimensions represent.

Dataset & boilerplate

Why this data fits

The delta-and-stack operation in steps 3–4 is the exact pattern `librosa.feature.delta()` uses internally, and `vstack([base, delta])` is how you'd build the combined matrix for `librosa` Exercise 9.

Arrays you'll work with

- `base` — (3, N) base feature matrix
- `delta` — (3, N-1) first difference
- `combined` — (6, N) base + delta stacked

Boilerplate — NASA Exoplanet Archive — same load as Ex 01

Reuse the clean arrays. The delta operation in step 3 is the key new concept.

```
# Reuse clean arrays from Exercise 04

valid = np.isfinite(mass) & np.isfinite(radius) & np.isfinite(period) &
np.isfinite(ecc)

m = mass[valid]; r = radius[valid]
p = period[valid]; e = ecc[valid]


# Build a 3xN base matrix
base = np.vstack([m, r, p])          # shape (3, N)
print('base shape:', base.shape)


# Delta (first difference along time axis):
delta = np.diff(base, axis=1)        # shape (3, N-1)
print('delta shape:', delta.shape)  # note: one fewer column


# To stack base + delta you need equal column counts.
# Pad delta by repeating its last column:
delta_padded = np.hstack([delta, delta[:, -1:]]) # shape (3, N)
combined = np.vstack([base, delta_padded])        # shape (6, N)
print('combined shape:', combined.shape)
```

12

Tier 4 — Numerical methods

Logarithms, decibels, and amplitude scaling

NumPy concepts used

```
np.log()    np.log10()    np.log2()    np.exp()    amplitude_to_db pattern    np.clip()    np.maximum()
```

Objective

Apply log transforms to compress dynamic range — the same conversion `librosa.amplitude_to_db()` performs, and the reason spectrograms are plotted in dB rather than raw magnitude.

Instructions

56. Compute `np.log10(s_lum[valid])` for valid stellar luminosities. Compare the range before and after the log transform by printing min, max, and std.
57. Simulate `librosa.amplitude_to_db()` formula on the exoplanet mass array: `db = 20 * np.log10(np.maximum(mass_clean, 1e-10))`. This converts amplitude to decibels.
58. Use `np.clip()` to cap the mass array at a maximum of 10 Earth masses (`np.clip(mass_clean, 0, 10)`). Then apply the log transform to the clipped version. Compare to the unclipped result.
59. Reverse the dB conversion: recover the original `mass_clean` from `db` using the inverse formula: `recovered = 10 ** (db / 20)`. Verify with `np.allclose()`.
60. Plot (using `matplotlib`) the mass distribution on a linear scale vs. a log scale using `np.log10(mass_clean)`. Observe which scale better reveals structure in the data.

Dataset & boilerplate

Why this data fits

Planetary mass spans several orders of magnitude — from sub-Earth to thousands of Jupiter masses — making it a natural analog for audio amplitude. The log-compression exercise here is the same mathematical operation as `librosa.amplitude_to_db()`.

Arrays you'll work with

- `mass_clean` — raw amplitude-like values (large dynamic range)
- `db` — log-transformed version
- `s_lum` — stellar luminosity (already log-like in solar units)

Boilerplate — NASA Exoplanet Archive — same load as Ex 01

Reuse `mass_clean` and `s_lum` from previous exercises.

```
# Reuse mass_clean from Exercise 04:
valid = np.isfinite(mass)
mass_clean = mass[valid]

# Simulate amplitude_to_db — the exact librosa formula:
# librosa.amplitude_to_db(S)  $\equiv 10 * \log_{10}(S^{*2})$ 
#  $\equiv 20 * \log_{10}(S)$  (for amplitude, not power)
ref = 1e-10 # floor to avoid log(0)
db = 20 * np.log10(np.maximum(mass_clean, ref))

print('mass range (linear):', mass_clean.min(), '-', mass_clean.max())
print('mass range (dB):', db.min().round(2), '-', db.max().round(2))
```

13

Tier 4 — Numerical methods

Cross-similarity matrix construction

NumPy concepts used

`np.dot()` on matrices L2 normalization broadcasting for outer norms `np.linalg.norm(axis=)`
 pairwise distance logic `np.fill_diagonal()`

Objective

Build a full pairwise cosine similarity matrix between two sets of vectors — the operation at the heart of `librosa.segment.cross_similarity()`.

Instructions

- Take two subsets of stars from the stellar mat matrix: the first 200 columns (`mat_A`) and the next 200 columns (`mat_B`). Each is shape (5, 200).
- L2-normalize each matrix column-wise: divide each column by its Euclidean norm. Use `np.linalg.norm(mat_A, axis=0, keepdims=True)` and broadcast divide.
- Compute the (200, 200) cross-similarity matrix using `np.dot(mat_A_norm.T, mat_B_norm)`. Each cell (i,j) is the cosine similarity between star i and star j.
- Verify: diagonal values of a self-similarity matrix (`mat_A` vs `mat_A`) should be 1.0. Use `np.allclose(np.diag(self_sim), 1.0)`.
- Find the most similar star in `mat_B` for each star in `mat_A`: use `np.argmax(cross_sim, axis=1)`. This returns 200 indices — the column-wise argmax is the same operation librosa uses to find the closest matching frame.

Dataset & boilerplate

Why this data fits

This is `librosa.segment.cross_similarity()` implemented from scratch. The two 200-star subsets play the role of two audio segments; the (200, 200) similarity matrix is exactly what `specshow` renders as a bright-diagonal cross-similarity plot.

Arrays you'll work with

- `mat_A, mat_B` — (5, 200) feature submatrices
- `A_n, B_n` — column-normalized versions
- `cross_sim` — (200, 200) cosine similarity matrix

Boilerplate — HYG Stellar Database — stellar feature matrix

Reuse `mat` from Exercise 09. Build `mat_A` and `mat_B` by slicing column ranges.

```
# Reuse mat from Exercise 09 (shape: 5 × N_valid)

# Normalize mat row-wise first (so scale differences don't dominate):
row_m = np.mean(mat, axis=1, keepdims=True)
row_s = np.std(mat, axis=1, keepdims=True)
mat_norm = (mat - row_m) / (row_s + 1e-10) # shape (5, N_valid)

# Slice two subsets for cross-similarity:
mat_A = mat_norm[:, :200] # shape (5, 200)
mat_B = mat_norm[:, 200:400] # shape (5, 200)

# L2-normalize columns (each star vector becomes unit length):
def l2_norm_cols(M):
    norms = np.linalg.norm(M, axis=0, keepdims=True) # shape (1, 200)
    return M / (norms + 1e-10)

A_n = l2_norm_cols(mat_A) # shape (5, 200), unit-norm columns
B_n = l2_norm_cols(mat_B)
print('A_n shape:', A_n.shape)
```

14

Tier 4 — Numerical methods

Accumulated cost matrix and path tracing (DTW logic)

NumPy concepts used

`np.cumsum()` `np.minimum()` `np.inf` `np.zeros()` `np.flipud()` `np.unravel_index()` nested loop with NumPy ops

Objective

Build the dynamic programming table at the core of Dynamic Time Warping — the algorithm behind

`librosa.sequence.dtw()` in Exercise 14 of the `librosa` series.

Instructions

66. Compute a simple (50, 50) cost matrix: $C[i,j]$ = Euclidean distance between column i of `mat_A[:,50]` and column j of `mat_B[:,50]`. Use a nested loop and `np.linalg.norm()`.
67. Initialize a (50, 50) accumulated cost matrix `D` filled with `np.inf` using `np.full((50,50), np.inf)`. Set `D[0,0] = C[0,0]`.
68. Fill the first row and first column of `D` using cumsum-style logic: $D[0,j] = D[0,j-1] + C[0,j]$ and $D[i,0] = D[i-1,0] + C[i,0]$.
69. Fill the rest of `D` with the DTW recurrence: $D[i,j] = C[i,j] + \min(D[i-1,j], D[i,j-1], D[i-1,j-1])$. Use `np.minimum()` where possible.
70. Trace the optimal warping path back from `D[49,49]` to `D[0,0]` by greedily choosing the minimum neighbor at each step. Collect the (i,j) pairs into a list. This is your warping path.

Dataset & boilerplate

Why this data fits

Building DTW manually before using `librosa.sequence.dtw()` is the most effective way to understand what the warping path means. The cost matrix shape, the accumulated cost matrix, and the path-tracing step are all directly visible here.

Arrays you'll work with

- `A50, B50` — (5, 50) submatrices used as DTW inputs
- `C` — (50, 50) pairwise cost matrix
- `D` — (50, 50) accumulated cost matrix

Boilerplate — HYG Stellar Database — same load as Ex 09

Reuse `mat_A` and `mat_B` (first 50 columns of each) from Exercise 13. The DTW here is the same algorithm `librosa` runs — just made transparent.


```
# Reuse A_n and B_n from Exercise 13

# Work with the first 50 columns for speed:
A50 = A_n[:, :50]    # shape (5, 50)
B50 = B_n[:, :50]    # shape (5, 50)


# Step 1: build cost matrix
N = 50
C = np.zeros((N, N))
for i in range(N):
    for j in range(N):
        C[i, j] = np.linalg.norm(A50[:, i] - B50[:, j])

print('C shape:', C.shape)
print('C min/max:', C.min().round(3), '/', C.max().round(3))

# Steps 2-5 build on C - follow instructions above
```

15

Tier 5 — Full pipeline

Full analysis pipeline: normalization → similarity → comparison

NumPy concepts used

```
all prior concepts combined  np.corrcoef()  np.cov()  np.linalg.norm()  np.allclose()
np.nan_to_num()
```

Objective

Connect all the concepts from Exercises 01–14 into a single analysis pipeline — the NumPy equivalent of librosa Exercise 15 (the full audio analysis pipeline).

Instructions

71. Load both datasets. Build a (5, 500) stellar feature matrix and a (5, 500) exoplanet feature matrix (mass, radius, period, ecc, dist — all clean). Normalize both with axis=1, keepdims=True.
72. Compute a (500, 500) cross-similarity matrix between the stellar and exoplanet matrices using the L2-normalized column dot-product from Exercise 13.
73. Compute a (500, 500) self-similarity matrix for the stellar matrix alone. Verify the diagonal is 1s and the matrix is symmetric using np.allclose(sim, sim.T).
74. On the cross-similarity matrix, find the best matching exoplanet for each star using np.argmax(axis=1). Then compute the distribution of similarity scores at those best-match positions using np.mean() and np.std().
75. Run your simplified DTW from Exercise 14 on a (30, 30) submatrix of the cross-similarity matrix (convert it to a cost matrix using 1 - sim). Extract the warping path, print its length, and plot both the cost matrix and the path using matplotlib from what you practiced in the matplotlib exercises.

Dataset & boilerplate

Why this data fits

This is the capstone exercise. Every array operation from Exercises 01–14 appears here. The cross-similarity between stellar features and exoplanet features is physically meaningful: it asks 'which exoplanet's host star most resembles this nearby star?'

Arrays you'll work with

- `e_mass ... e_dist` — 5 exoplanet property arrays
- `s_mag ... s_dist` — 5 stellar property arrays
- Both become (5,500) matrices for the pipeline

Boilerplate — Both datasets — NASA Exoplanet Archive + HYG stellar database

Both datasets need to be loaded. Run both boilerplates below. This is the only exercise requiring both sources.

```

# — Dataset 1: NASA Exoplanet Archive —————
import numpy as np, pandas as pd

TAP = 'https://exoplanetarchive.ipac.caltech.edu/TAP/sync'

Q = ('?query=select+pl_masse,pl_rade,pl_orbper,pl_orbeccen,sy_dist'
     '+from+pscomppars+where+pl_masse+is+not+null'
     '+and+pl_rade+is+not+null+and+sy_dist+is+not+null'
     '&format=csv')

df_exo = pd.read_csv(TAP + Q)

df_exo = df_exo[~df_exo.iloc[:,0].astype(str).str.startswith('#')]

e_mass = df_exo['pl_masse'].to_numpy(float)
e_rad  = df_exo['pl_rade'].to_numpy(float)
e_per  = df_exo['pl_orbper'].to_numpy(float)
e_ecc  = df_exo['pl_orbeccen'].to_numpy(float)
e_dist = df_exo['sy_dist'].to_numpy(float)


# — Dataset 2: HYG Stellar Database —————
HYG = ('https://raw.githubusercontent.com/astronexus/'
      'HYG-Database/main/hyg/v3/hygdata_v3.csv')

df_hyg = pd.read_csv(HYG)

df_hyg = df_hyg[df_hyg['dist'] < 100000].reset_index(drop=True)

s_mag  = df_hyg['mag'].to_numpy(float)
s_abs  = df_hyg['absmag'].to_numpy(float)
s_ci   = df_hyg['ci'].to_numpy(float)
s_lum  = df_hyg['lum'].to_numpy(float)
s_dist = df_hyg['dist'].to_numpy(float)

print('Exoplanets:', len(e_mass), '| Stars:', len(s_dist))

```